Optimizing Monte-Carlo VaR for Structured Mortgages

Laurent Motais de Narbonne

Abstract

This article presents an innovative approach to measure VaR that combines the use of state-of-the-art random generators with Monte-Carlo simulation using antithetic variables, to capitalise on the respective strengths of these approaches. The results presented here on a structured mortgage transaction with prepayments show that these techniques, combined with a flexible framework in terms of distributions chosen to model the risk factors, significantly enhance the performance and precision of the Monte-Carlo engine.

1 Comparing Different Approaches in Terms of Speed and Accuracy

A naïve, or plain, Monte-Carlo simulation for VaR performs full trade revaluations of an entire portfolio under thousands of random scenarios to approximate the profit and loss distribution. The trade revaluation process is computationally intensive, which prevents Monte-Carlo from being a viable tool to measure risk on its own. Given that our goal is to obtain VaR using the best approach in terms of speed and accuracy, our approach uses Monte-Carlo simulation with antithetic variables, following this step-by-step process:

Random factors path generation. This involves generating pseudo-random scenarios for the risk factors (interest rates, bond and equity prices, foreign exchange rates, volatilities, etc.). The definition of risk factors is a critical input to the analysis. Recognising that during crises, historical measures of volatilities and correlations break down, it is important to supplement VaR with event risk analysis. Our approach uses different random generators (Random Generator 1, 2, 3, 4, Quick and Dirty, Mersenne Twister, and Latin Hypercube).

Monte-Carlo VaR calculation. The full valuation profits and losses for all scenarios are computed, and the Monte-Carlo VaR is obtained by assessing the α-percentile based on the original number of scenarios generated, where $(1 - \alpha)$ is the confidence level.

2 Variance Reduction

Monte-Carlo results explicitly include a calculated error term. Our method uses a variance reduction technique – antithetic variables – combined with different random generators, to reduce the Monte-Carlo error term. By reducing the error term, risk managers may obtain either a higher degree of precision for the same number of random scenarios, or improved performance by speeding up Monte-Carlo convergence for the same specified error.

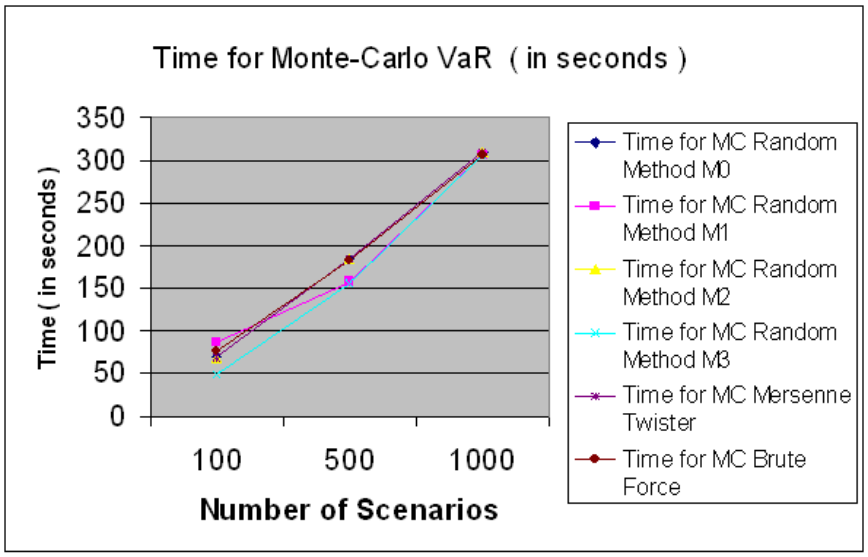


Figure 1: Time vs Number of scenarios for Monte-Carlo VaR

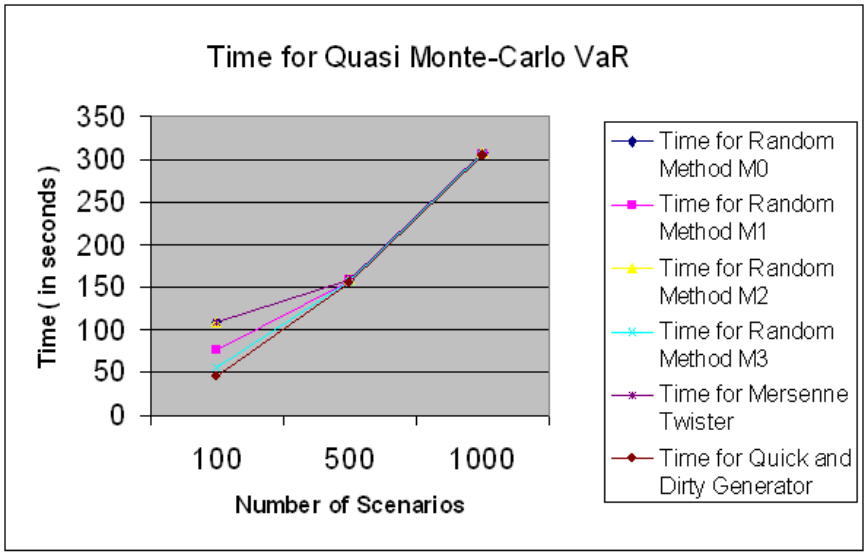


Figure 2: Time vs Number of scenarios for Quasi Monte-Carlo VaR

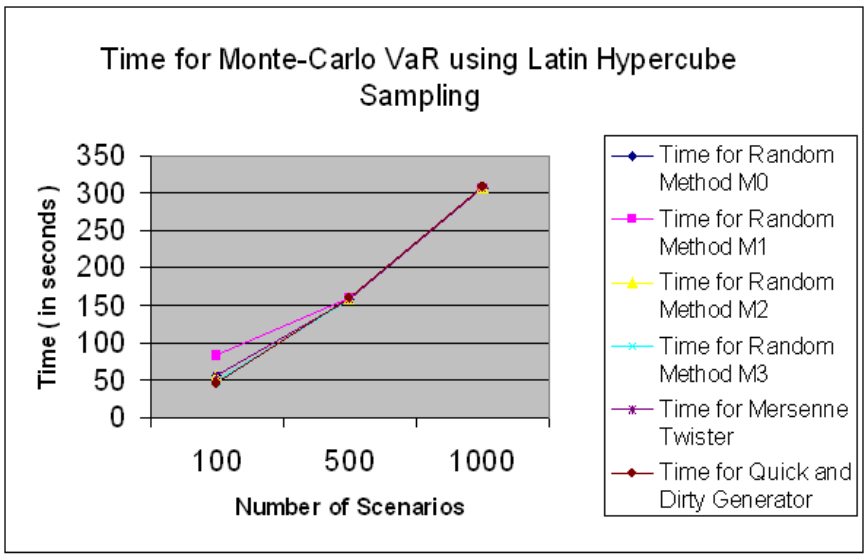


Figure 3: Time vs Number of scenarios for Monte-Carlo VaR using Latin Hypercube Sampling

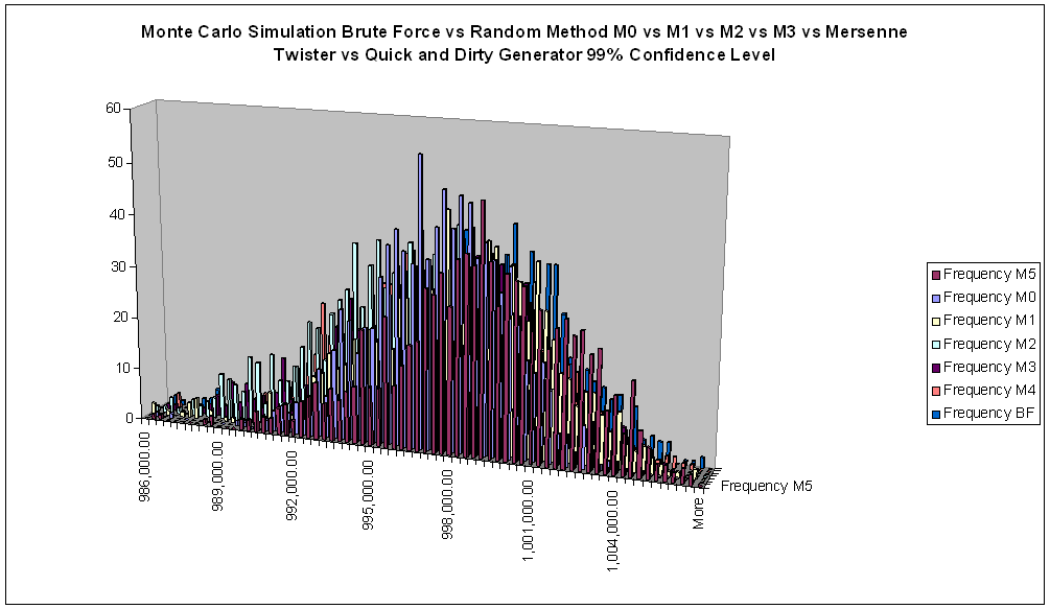


Figure 4: Plain Monte-Carlo vs Antithetic Variables by Random Generator

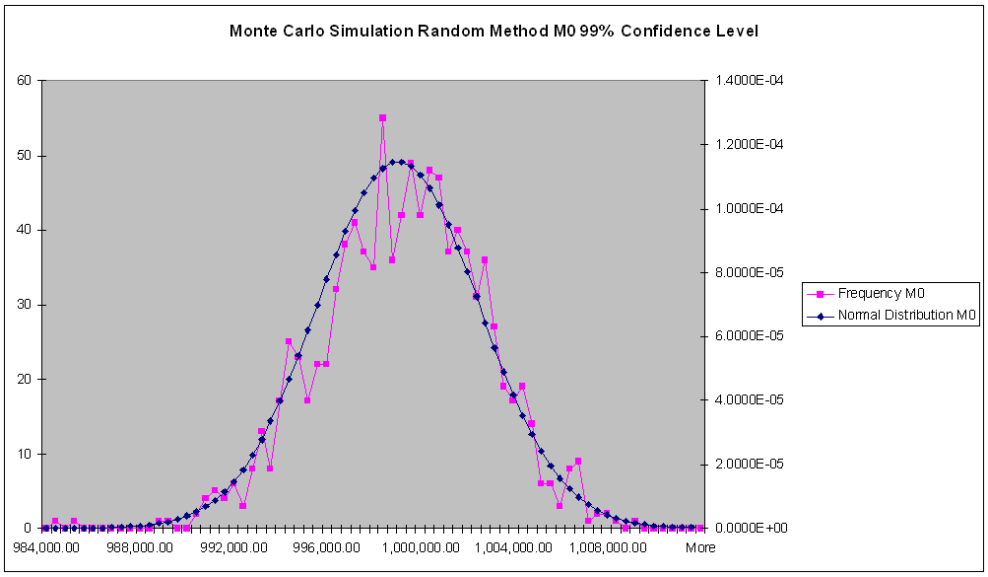


Figure 5: Monte-Carlo simulation: Frequency vs Normal Distribution using Random Method 0

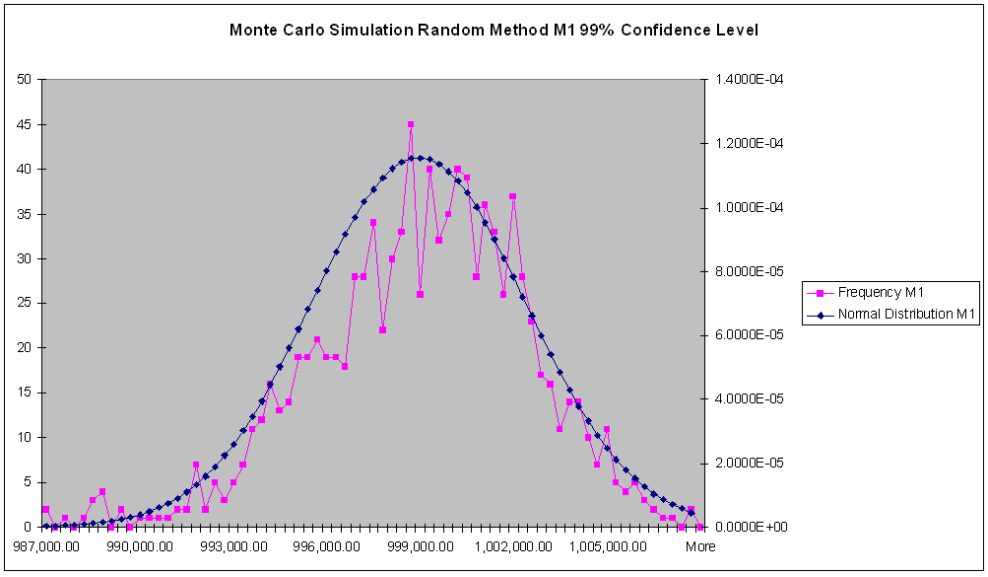


Figure 6: Monte-Carlo simulation: Frequency vs Normal Distribution using Random Method 1

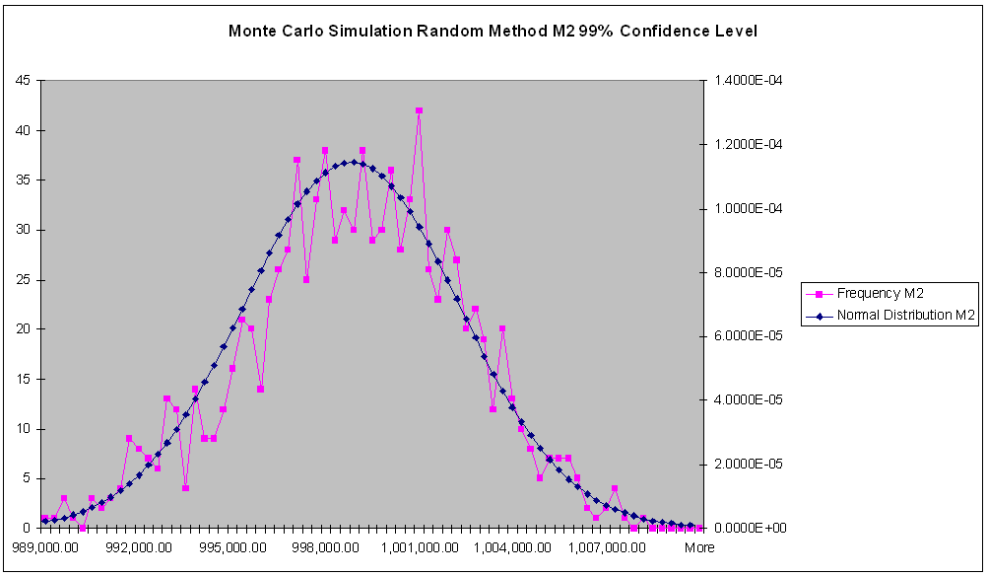


Figure 7: Monte-Carlo simulation: Frequency vs Normal Distribution using Random Method 2

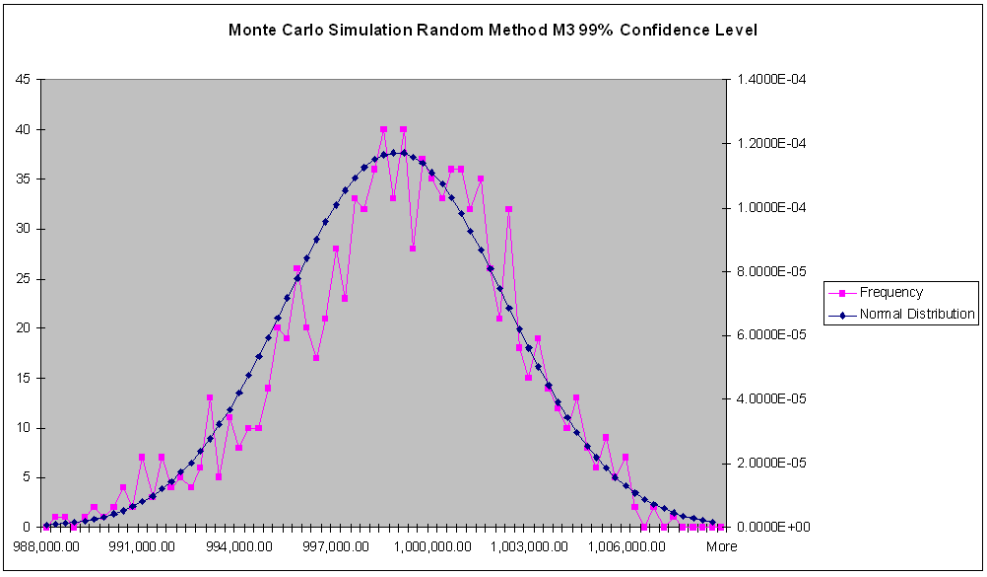


Figure 8: Monte-Carlo simulation: Frequency vs Normal Distribution using Random Method 3

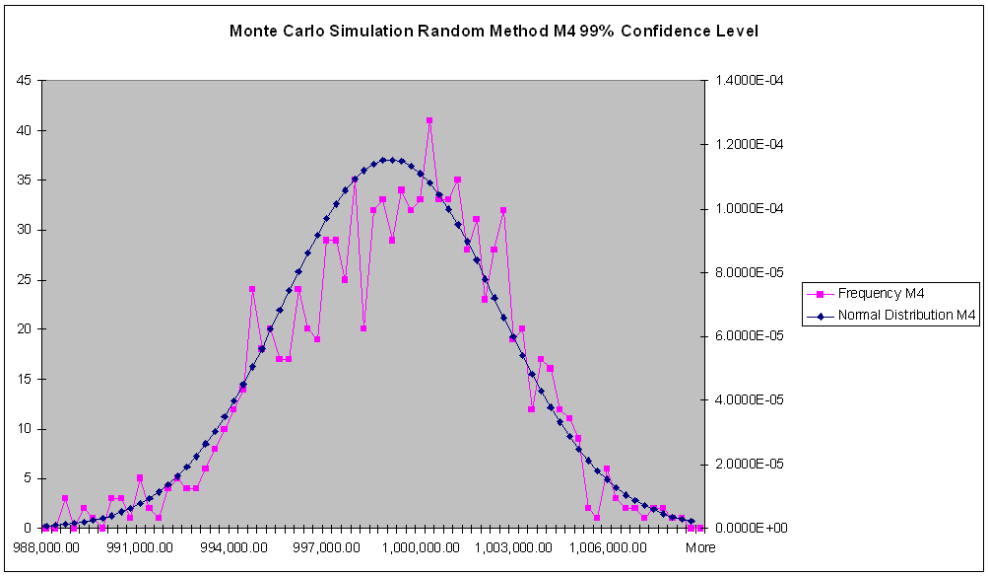


Figure 9: Monte-Carlo simulation: Frequency vs Normal Distribution using Mersenne Twister

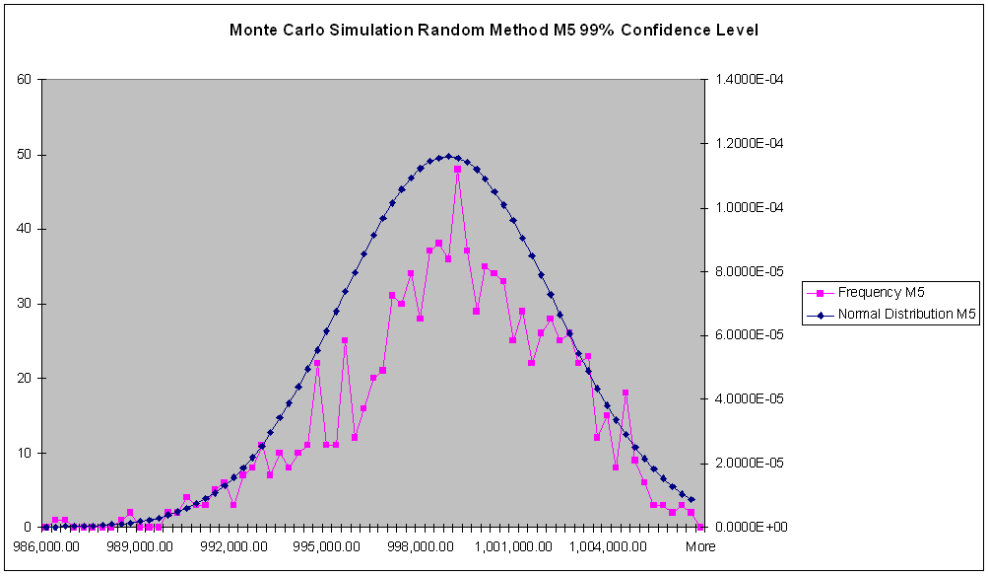


Figure 10: Monte-Carlo simulation: Frequency vs Normal Distribution using Quick and Dirty Generator

Antithetic Variables

By using antithetic variables, the variance of the estimator is reduced by a factor $\frac{1}{2}\rho$, where ρ is the correlation. For most test cases, using antithetic variables for the calculation of Monte-Carlo VaR makes the simulation much faster.

One of the most attractive features of antithetic variables is that they can be easily applied to the full distribution, yielding a much smoother curve than a plain Monte-Carlo approach. Figure 4 shows a comparison of a plain Monte-Carlo approach with Monte-Carlo using antithetic variables, for 1000 scenarios.

Monte-Carlo Results

To illustrate the beneficial effects on performance and precision of the VaR estimate provided by our Monte-Carlo approach, VaR results for a 99% confidence level and a one-day time horizon are generated for a structured mortgage with prepayments.

The fact that this type of instrument has a non-monotonic pay-off and exhibits significant convexity is crucial to demonstrate the power of a Monte-Carlo approach combined with antithetic variables and state-of-the-art random generators, given that, for this type of instrument, parametric VaR is not sufficient to estimate VaR.

As Figure 1 shows, performance times can be significantly reduced when using a Monte-Carlo simulation approach combined with antithetic variables and Random Method 2 or 3, rather than a plain Monte-Carlo approach. Performance times could have been reduced further in a distributed processing environment with multiple servers – the desired configuration for generating global VaR.

Figure 1 compares the plain Monte-Carlo approach results with the optimised Monte-Carlo VaR results using antithetic variables and different random number generators. These results demonstrate the improvement in precision obtained by generating VaR using an optimised Monte-Carlo VaR approach.

3 Conclusion

The results displayed above clearly validate the use of Monte-Carlo simulation combined with variance reduction techniques and optimised random number generators, and demonstrate the performance gains obtained with this combined approach.

With this approach, risk managers have complete flexibility to determine the extent to which performance optimisations are used when generating VaR results. VaR obtained using the optimised Monte-Carlo simulation may be back-tested against a plain Monte-Carlo simulation, providing risk managers with an empirical view on the accuracy of VaR measures, and the resulting ability to allocate capital and risk limits confidently and optimally.

A Variance Reduction

Plain Monte-Carlo Approach

For a given level x_a of actual profit and loss, the percentile is estimated using a sum of indicator functions:

$$\overline{F}_a(x_a) = \frac{\sum_{i=1}^{N} \mathbf{1}_{\{a_i \leq x_a\}}}{N} \tag{1}$$

where a_i is the actual profit and loss for scenario i. For a sufficiently large number of scenarios, this estimator is approximately normally distributed, with mean equal to the true percentile

and variance approximately equal to:

$$\overline{\sigma}^2\big(\overline{F}_a(x_a)\big) = \frac{\overline{F}_a(x_a)\big(1-\overline{F}_a(x_a)\big)}{N-1} \tag{2}$$

Note. To find a confidence interval for the VaR, one needs to map the confidence interval on the percentile back to the profit and loss space, using the cumulative distribution function.

Antithetic Variables

The method of antithetic variables is based on the observation that if X follows a standard normal distribution, then so does $-X$.

If we model the actual profit and loss a_i for each scenario i as:

$$a_i = \delta_i z_i + \frac{1}{2}\lambda_i z_i^2, \qquad z_i \in \mathbb{R}^N \text{ i.i.d. standard normal}$$

we can rewrite the estimator $\overline{F}_a(x_a)$ as:

$$\overline{F}_a(x_a) = \frac{\sum_{i=1}^N f(z_i)}{N}, \qquad f(z_i) = \mathbf{1}_{a_i \le x_a} = \mathbf{1}_{\left\{\delta_i z_i + \frac{1}{2}\lambda_i z_i^2 \le x_a\right\}}$$

Similarly, $\widetilde{F}_a(x_a) = \dfrac{\sum_{i=1}^N f(-z_i)}{N}$ is also an unbiased estimator of the same percentile at level x_a. Therefore:

$$F_{av}(x_a) = \frac{\overline{F}_a(x_a) + \widetilde{F}_a(x_a)}{2}$$

is also an unbiased estimator of the same percentile.

The intuition behind the antithetic method is that the random inputs obtained from the collection of antithetic pairs are more regularly distributed than a collection of $2N$ independent samples.

The standard error of this estimator is:

$$\sigma_{av}^2 = \text{Variance}\big(F_{av}(x_a)\big) = \frac{\text{Variance}\big(\overline{F}_a(x_a)\big) + \text{Variance}\big(\widetilde{F}_a(x_a)\big)}{4} + \frac{\text{Cov}\big(\overline{F}_a(x_a), \widetilde{F}_a(x_a)\big)}{2}$$

$\text{Cov}\big(\overline{F}_a(x_a), \widetilde{F}_a(x_a)\big)$ being negative, the variance of the initial estimator $\overline{F}_a(x_a)$ is reduced by $\dfrac{\text{Cov}(\overline{F}_a(x_a), \widetilde{F}_a(x_a))}{2}$.

B Random Number Generators: Ran0, Ran1, Ran2, Ran3, and the Quick and Dirty Generator

Ran0

Ran0 is defined using the simple multiplicative congruential algorithm:

$$I_{j+1} = a\, I_j \bmod m, \qquad a = 7^5 = 16807, \qquad m = 2^{31} - 1 = 2147483647$$

This generator has, in subsequent years, passed theoretical tests and accumulated a large amount of successful use. Although not perfect, it is a good minimal standard against which other generators should be judged. For ease of implementation, m is decomposed as $m = aq + r$, with $q = 127773$ and $r = 2836$. The routine Ran0 is a minimal standard, satisfactory for the majority of applications.

Ran1

Ran1 uses the minimal standard for its random value, but shuffles the output to remove low-order serial correlations. Ran1 passes those statistical tests that Ran0 is known to fail. In fact, no statistical test is known to make Ran1 fail, except when the number of calls becomes of the order of the period m, say $> 10^8 \approx m/20$.

Ran2

Combining the following two generators:

$$I_{j+1} = a_1 I_j \bmod m_1, \qquad a_1 = 40014,\ q_1 = 53668,\ r_1 = 12211,\ m_1 = 2147483563$$

$$I_{j+1} = a_2 I_j \bmod m_2, \qquad a_2 = 40692,\ q_2 = 52774,\ r_2 = 3791,\ m_2 = 2147483399$$

defines a new random number generator, denoted Ran2.

Ran3

Ran3 is not based on the linear congruential method, but on a subtractive method.

Quick and Dirty Random Number Generator

Defined as:

$$I_{j+1} = a\, I_j + c, \qquad a = 1664525, \qquad c = 1013904223$$

C Quasi-Monte Carlo Methods and Latin Hypercube Sampling

Quasi-Monte Carlo methods rely on sequences of n-tuples that fill an n-space more uniformly than uncorrelated random points – so-called *quasi-random sequences*. Examples include Halton's sequence and Sobol's sequence.

The technique of Latin Hypercube Sampling is used when sampling an N-dimensional space, exceedingly sparsely, at M points. It can be thought of as an extension of antithetic variables, creating negative dependence among more than two samples.

Limits of Quasi-Monte Carlo Methods

Randomised Quasi-Monte Carlo (RQMC) or Latin Hypercube Sampling (LHS) methods can improve results for stochastic simulations only when the number of risk factors is limited (2 to 4). This is due to the so-called "curse of dimensionality", by which non-random sampling techniques cannot outperform random sampling (i.e. regular MC) once the number of independent dimensions (or parameters) of the calculation becomes too large. In real-life applications, this limit is usually reached at about three independent dimensions.

References

References

[1] Fishman, G. (1995). *Monte Carlo, Concepts, Algorithms, and Applications.* Springer-Verlag.

[2] Glasserman, P., Heidelberger, P., and Shahabuddin, P. (2000). Efficient Monte Carlo Methods for Value at Risk. IBM Research Paper RC21723.

[3] Press, W.H., Teukolsky, S.A., Vetterling, W.T., and Flannery, B.P. (1992). *Numerical Recipes in C*, 2nd edition. Cambridge University Press.